

PLPP Assignment 3

Matvii Kovalyshyn

Git repo link:

<https://github.com/VY-Assignments/text-editor-matvii-kovalyshyn>

Program explanation

- **OOP design:** describe your class hierarchy — Cipher base class, CaesarCipher and VigenereCipher as derived classes, why each field is private

I have Cipher base class, which implements shared logic and structure for both CaesarCipher and VigenereCipher. Each field is private due to protection from random key changes. Key must be set only with creation of an object.

- **C library API:** explain your design choices — why the public API uses C functions instead of exporting C++ classes directly

C API has more understandable names of class and its methods. Also, C API is more stable.

- **Virtual dispatch:** walk through what happens when cipher_encrypt is called — how the vtable resolves the correct implementation at runtime

In main we call cipher_encrypt. Inside this function we create an object of the base class Cipher. This object has a field of virtual pointer – vptr, which points to the table of class methods. As for encrypt, it has 1 index, so we call the first method of the class.

- **extern "C":** explain why it is needed and what name mangling would look like without it (show a dumpbin/nm example)

extern “C” means that we will compile it as a C language. It is necessary because C API is more stable and has clear names. Without extern “c” we would have messy method names with a lot of symbols at the beginning and at the end.

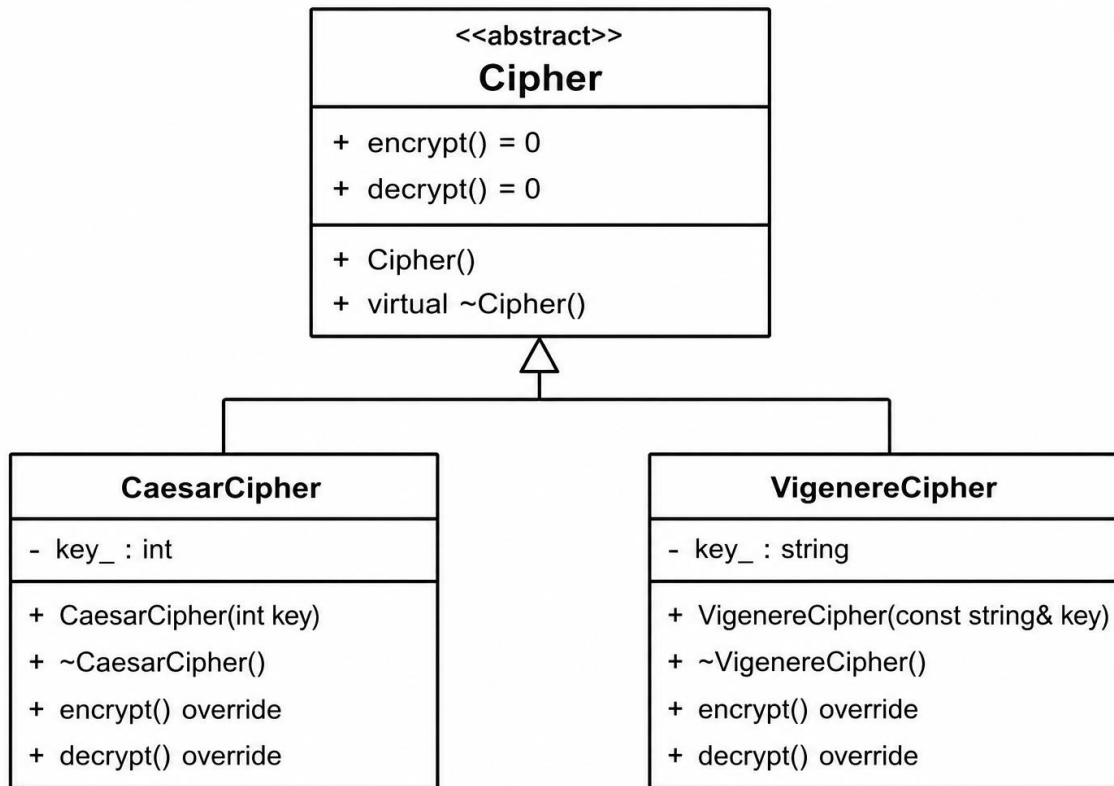
- **Memory management:** who allocates and who frees — cipher_create_*, cipher_destroy, cipher_free

cipher_create* allocates memory. cipher_destroy frees memory of cipher object. cipher_free frees memory of strings used for storing encrypted and decrypted texts.

Class diagram

Sketch your class hierarchy (hand-drawn or tool-generated). Must show:

- Cipher (abstract) with encrypt() and decrypt() as pure virtual methods
- CaesarCipher and VigenereCipher as derived classes with their private key_ fields
- Arrows indicating inheritance



Test plan — one screenshot per scenario

- **Caesar encrypt:** key 3, input “Hello, World!” → output “Khoor, Zruog!”

```
Choose a cipher: caesar(1) or vigenere(2). Exit(0).
-> 1
Choose a key: 3

Enter text: Hello, World!

Caesar.
encrypted = Khoor, Zruog!
decrypted = Hello, World!
```

- **Caesar round-trip:** encrypt then decrypt returns the original message

```
Choose a cipher: caesar(1) or vigenere(2). Exit(0).
-> 1
Choose a key: 3

Enter text: Hello, World!

Caesar.
encrypted = Kloor, Zruog!
decrypted = Hello, World!
```

- **Vigenère encrypt:** keyword KEY, input “Hello, World!” → output “Rijvs, Uyvjn!”

```
Choose a cipher: caesar(1) or vigenere(2). Exit(0).
-> 2
Choose a key: key

Enter text: Hello, World!

Vigenere.
encrypted = Rijvs, Uyvjn!
decrypted = Hello, World!
```

- **Vigenère round-trip:** encrypt then decrypt returns the original message

```
Choose a cipher: caesar(1) or vigenere(2). Exit(0).
-> 2
Choose a key: key

Enter text: Hello, World!

Vigenere.
encrypted = Rijvs, Uyvjn!
decrypted = Hello, World!
```

- **Library loads at runtime:** show successful dlopen/LoadLibrary and function pointer resolution

```
◦ [kumasik@archlinux Encryption]$ ./main
[DEBUG] Dynamic Library Loading Success!
-> libcipher.so handle address: 0x559c5cbe5040
-> cipher_create_caesar fn pointer: 0x7f7bbb51221b
-> cipher_encrypt fn pointer: 0x7f7bbb5123c2
```

– **Edge cases (from quality acceptance criteria):**

- Non-letter characters (spaces, punctuation) pass through unchanged

```
Choose a cipher: caesar(1) or vigenere(2). Exit(0).
-> 2
Choose a key: key

Enter text: Hello, World!

Vigenere.
encrypted = Rijvs, Uyvjn!
decrypted = Hello, World!
```

- Key wraps correctly at alphabet boundary ($Z + 1 = A$)

```
Choose a cipher: caesar(1) or vigenere(2). Exit(0).
-> 1
Choose a key: 3

Enter text: xyz

Caesar.
encrypted = abc
decrypted = xyz
```

- Vigenère key cycles correctly over a message longer than the key

```

Choose a cipher: caesar(1) or vigenere(2). Exit(0).
-> 2
Choose a key: key

Enter text: Hello, World!

Vigenere.
encrypted = Rijvs, Uyvjn!
decrypted = Hello, World!

```

- No memory leaks — run under Valgrind (Linux) or AddressSanitizer and show clean output

```

==54839==
==54839== HEAP SUMMARY:
==54839==    in use at exit: 0 bytes in 0 blocks
==54839==   total heap usage: 18 allocs, 18 frees, 80,493 bytes allocated
==54839==
==54839== All heap blocks were freed -- no leaks are possible
==54839==
==54839== For lists of detected and suppressed errors, rerun with: -s
==54839== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

```

Git history screenshot (git log / GitHub / gitk)

